

Documentación técnica — Portafolio Óptimo (Grupo 3)

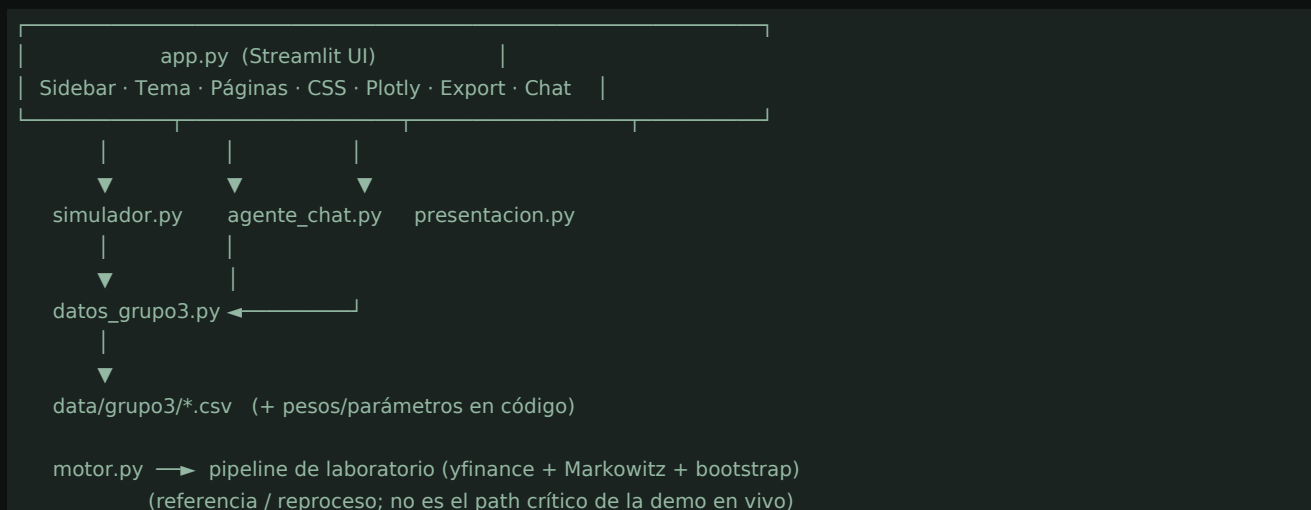
Documento de apoyo para explicar cómo se construyó la aplicación, la arquitectura, el flujo de datos y el rol de cada módulo. Complementa el README.md de la raíz del repositorio.

1. Contexto del proyecto

Campo	Detalle
Curso	Programación para ejecutivos — Maestría Manageme
Producto	App web de apoyo a un comité de inversión
Problema	Presentar un portafolio óptimo (Markowitz / Máx.
Entregable técnico	Repositorio GitHub + app Streamlit (local y/o Co

El análisis cuantitativo original se realizó en un notebook Colab (descarga de precios, optimización, riesgo, bootstrap). La app empaqueta esos resultados y permite explorarlos de forma ejecutiva: no exige volver a correr Yahoo Finance en cada sesión de demo.

2. Arquitectura general



Principio de diseño

Separar UI, simulación en vivo y motor de laboratorio.

- La UI no implementa optimización Markowitz en cada clic.
- El simulador ajusta escenarios sobre métricas y pesos calibrados.
- El motor completo queda disponible para auditar o regenerar el análisis.

3. Flujo de una sesión de usuario

1. Streamlit arranca `app.py` y configura `page_config` (layout wide, sidebar).
2. Se inicializa `session_state` (tema, pagina, `demo_paso`, parámetros del simulador).
3. El sidebar muestra navegación (`st.radio`) y toggle claro/oscuro (`st.segmented_control`).
4. Según parámetros (capital, perfil, horizonte, nº activos, moneda, confianza), se llama:

```
sim = simular(...)
```

5. Cada página consume el mismo diccionario `sim` (KPIs, pesos, clases, VaR, etc.).
6. En El asistente, `responder(pregunta, sim)` clasifica la intención y responde solo con datos de ese escenario.
7. En Resumen, se puede exportar CSV / TXT / PDF a partir del escenario actual.

4. Módulos — detalle

4.1 `app.py` — Interfaz

Responsabilidades:

- Navegación entre: Resumen, Simulador, Cartera, Riesgo, Desempeño, El asistente.
- Inyección de CSS (`_COMMON_CSS`, `_LIGHT_CSS`, `_DARK_CSS`) para sidebar, cards, hero y contraste.
- Tema claro/oscuro vía `session_state.tema`.
- Barra de parámetros del simulador (selectboxes).
- Gráficos Plotly (pesos, clases, comparador).
- Cards de decisión, elegibilidad y exportación.
- Recorrido guiado del comité (demo por pasos).

Helpers internos típicos: `money`, `pct`, `card_open` / `card_close`, `estado_semaforo`, `fig_pesos`, `fig_clases`, `informe_txt`.

4.2 `simulador.py` — Simulación en vivo

Entrada: capital, horizonte (meses), perfil (Máx. Sharpe / Conservador / Agresivo), confianza (VaR/CVaR), número de activos, moneda (S/ o US\$).

Proceso:

1. Toma métricas base anuales por perfil (BASE), alineadas al artefacto Colab.
2. Escala al horizonte: retorno lineal en $\sqrt{t}$, volatilidad con $\sqrt{t}$.
3. Ajusta Sharpe según número de activos (`_ajuste_n_activos`).
4. Calcula VaR / CVaR paramétricos (normal); en el caso canónico (Máx. Sharpe, 12 m, 20 activos) ancla VaR/CVaR a calibración Colab.
5. Recorta/renormaliza pesos (`_pesos_base` → `_recortar_activos`).
6. Agrega por clase de activo y convierte capital a USD si aplica ($TC = 3.39$).

Salida: diccionario `sim` con métricas, pesos, clases, escenarios de capital y comparación vs benchmark.

Ventaja: demo rápida, estable y reproducible en Streamlit Cloud (sin dependencia de red a Yahoo en runtime).

4.3 datos_grupo3.py — Fuente de verdad del mandato

- Constantes: CAPITAL, PARAMETROS_GRUPO3, PESOS_PORTAFOLIO.
- Lectura de CSV en data/grupo3/ (p. ej. cuadro resumen de escenarios).
- Función cargar_resultados(): escenarios adverso/esperado/favorable (portafolio y benchmark), asignación, clases, criterios de elegibilidad (p. ej. Sharpe ≥ 1.20 , VaR anual 95% $\leq 8\%$).

4.4 agente_chat.py — Asistente acotado

No usa API de LLM. Enfoque:

1. Normaliza texto (minúsculas, sin tildes).
2. Detecta saludo / gracias / fuera de dominio (deportes, clima, etc.) → rechazo educado.
3. Puntúa intenciones por keywords (elegibilidad, cartera, riesgo, sharpe, benchmark, etc.).
4. Genera respuesta plantilla con números del sim actual.
5. Ofrece *follow-ups* sugeridos.

Gobernanza: solo habla del portafolio / simulador del Grupo 3. Evita recomendaciones personalizadas fuera del marco académico.

4.5 presentacion.py — Capa narrativa

- frase_ejecutiva(...): texto de decisión para el comité.
- GLOSARIO, PASOS_RECORRIDO: apoyo didáctico.
- formatear_paso(...): contenido de cada paso del recorrido demo.

4.6 motor.py — Motor de laboratorio

Pipeline cuantitativo completo (referencia del trabajo original):

- Universo de tickers y clases de activo.
- Extracción de precios (yfinance).
- Regularización de covarianza.
- Optimización Markowitz (scipy.optimize.minimize) — p. ej. máximo Sharpe.
- VaR / CVaR, block bootstrap, max drawdown.
- Tablas de asignación y resumen por clase.
- ejecutar_analisis(params, progress=...) como orquestador.

Sirve para reproducir o auditar el análisis; la app de demo prioriza simulador.py + artefactos en data/grupo3/.

5. Datos

```
data/grupo3/  
├── 6_cuadro_resumen_escenarios.csv  
└── tabla_rentabilidad_riesgo_por_año.csv (+ variante de nombre)
```

Origen: exportes del notebook Colab del Grupo 3.

Si faltan CSV, cargar_resultados() falla con mensaje explícito indicando la ruta esperada.

assets/ contiene logos UTEC e ilustración del hero (SVG/PNG) usados solo en la UI.

6. UI y experiencia

Páginas

Página	Contenido principal
Resumen	Hero, KPIs, semáforo de elegibilidad, recomendac
Simulador	Controles y lectura de impacto en métricas
Cartera	Tabla + gráficos de pesos y clases
Riesgo	VaR/CVaR, volatilidad, vs benchmark
Desempeño	Retornos y escenarios de capital
El asistente	Chat + preguntas sugeridas

Tema claro / oscuro

- Estado en `st.session_state.tema`.
- CSS diferenciado para sidebar, controles, tablas y texto del hero.
- Objetivo: contraste legible (texto claro sobre fondo verde oscuro del sidebar).

Estilos

CSS embebido en `app.py` (no f-strings multilínea problemáticos en Python 3.14). Config base en `.streamlit/config.toml`.

7. Dependencias

Ver `requirements.txt`:

- `streamlit`
- `yfinance` (usado por `motor.py`)
- `numpy`, `pandas`, `scipy`
- `matplotlib`, `plotly`

Entorno recomendado: `virtualenv` local (`.venv`), no versionar el entorno en Git.

8. Decisiones de diseño (preguntas frecuentes)

¿Por qué no optimizar Markowitz en cada clic de la app?

Por estabilidad, velocidad y reproducibilidad en demos/cloud. Las métricas canónicas se anclan al artefacto Colab.

¿El chat es inteligencia artificial generativa?

Es un agente por reglas + matching de intenciones, acotado al dominio del portafolio. Facilita

gobernanza y evita alucinaciones fuera de tema.

¿Cuál es la diferencia entre motor.py y simulador.py?

- motor.py: pipeline completo de investigación.
- simulador.py: capa de producto para la UI.

¿Cómo se valida elegibilidad?

Reglas del mandato (p. ej. umbrales de Sharpe y VaR) evaluadas sobre resultados del escenario / CSV Grupo 3; la UI muestra semáforo y mensajes.

¿Qué se exporta?

Informes derivados del sim actual (CSV de pesos/métricas, TXT narrativo, HTML/PDF según botones de Resumen).

9. Cómo explicar el proyecto en 60 segundos

1. Optimizamos un portafolio Máximo Sharpe con datos históricos (Colab + motor.py).
2. Empaquetamos resultados y pesos en datos_grupo3 / CSV.
3. Construimos una app Streamlit para el comité: ver, simular, comparar riesgo y preguntar.
4. El asistente solo responde con números del escenario activo.
5. Todo queda en GitHub con README + esta documentación técnica.

10. Mapa rápido archivo → pregunta del evaluador

Te preguntan...	Mira...
¿Dónde está la UI?	app.py
¿Cómo cambian las métricas al mover sliders?	simulador.py → simular()
¿De dónde salen los pesos y el mandato?	datos_grupo3.py, data/grupo3/
¿Cómo responde el chat?	agente_chat.py → responder()
¿Dónde está Markowitz / bootstrap “de verdad”?	motor.py
¿Textos del comité / glosario?	presentacion.py
¿Cómo se instala?	README.md + requirements.txt

Grupo 3 · UTEC · Management Analytics & IA